

Informe Tarea 1

Florencia Aguirre, faguirre@usm.cl
Ozman Morales, omoralesd@usm.cl

1. Parte 1

La primera función simula la lectura de un sensor, devolviendo un valor entero en el rango [0, 255]. El diagrama de flujo de esta se muestra en la figura 1.

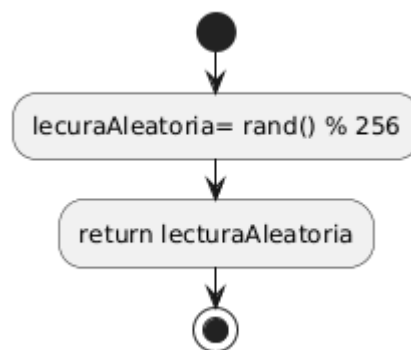


Figura 1: Función: `lecturaSensor()`.

La segunda función ocupa un arreglo para los sensores y un enum para los casos. Esta al ser llamada nos permite realizar acciones para cada sensor, según los casos en los que puede ser llamada. Siendo estos: iniciar, leer (`lecturaSensor`) y mostrar. El diagrama de flujo de esta se muestra en la figura 2.

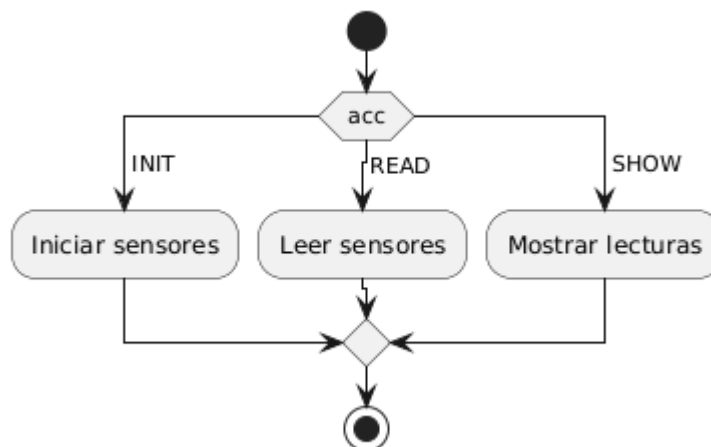


Figura 2: Función: `(int sensores[], Accion acc)`.

La función principal declara el arreglo sensores con nuestro número de sensores, después llama a nuestra segunda función en 3 ocasiones, cada una para una acción distinta, primero iniciar, luego leer y finalmente mostrar. El diagrama de flujo de esta se muestra en la figura 3

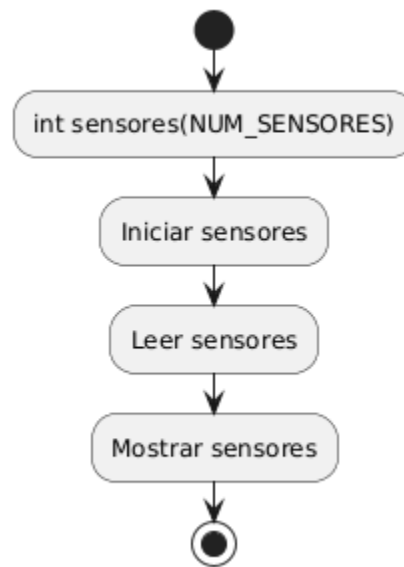


Figura 3: Función: main(int argc, char* argv[])

Parte 2

1.1. Hito 1

- A) ¿Qué es lo que retorna la función clock()?
: La función clock() retorna la cantidad de ticks (o ciclos) consumidos por el programa desde que comenzó su ejecución. Este valor representa tiempo de CPU en una unidad interna del sistema, no en segundos directamente.
- B) Qué unidades tienen las variables inicio y fin.
: Las variables inicio y fin tienen unidades ticks de reloj ya que almacenan directamente el valor retornado por clock()
- C) Por qué para obtener el tiempo total de duración del ciclo for se debe dividir la diferencia entre fin e inicio con el valor constante CLOCKS_PER_SEC.
: Porque la diferencia entre inicio-fin entrega la cantidad de ticks utilizados durante la ejecución del ciclo. Al dividir el valor obtenido por CLOCKS_PER_SEC, este se

convierte la cantidad de ticks a segundos, ya que esta constante indica cuántos ticks equivalen a un segundo.

- D) Explicar que hace (double) como prefijo de (fin - inicio):
: La función double dentro del código se encarga de convertir el resultado de la diferencia anteriormente calculada, a un valor tipo decimal para lograr una mayor precisión en los cálculos.
- E) Explicar para que sirve utilizar (double) como prefijo de (fin - inicio)
: Sirve para evitar una división de números enteros, logrando una mayor precisión en los cálculos

1.2. Hito 2

La función `cicloPrueba` utiliza un `enum` llamado `caso_t` para determinar qué tipo de operación será medida. Dependiendo del caso recibido, la función puede ejecutar un ciclo vacío, imprimir un espacio, imprimir un asterisco o imprimir un salto de línea. El diagrama de flujo de esta se muestra en la figura X.

Cada caso dentro del `switch` realiza el mismo procedimiento: primero se registra el tiempo inicial utilizando `clock()`, luego se ejecuta un ciclo `for` con `1e8` iteraciones realizando la acción correspondiente, posteriormente se registra el tiempo final y finalmente se calcula el tiempo promedio de ejecución en nanosegundos.

La función retorna el tiempo promedio obtenido para el caso seleccionado, permitiendo comparar el costo de ejecución de cada operación.

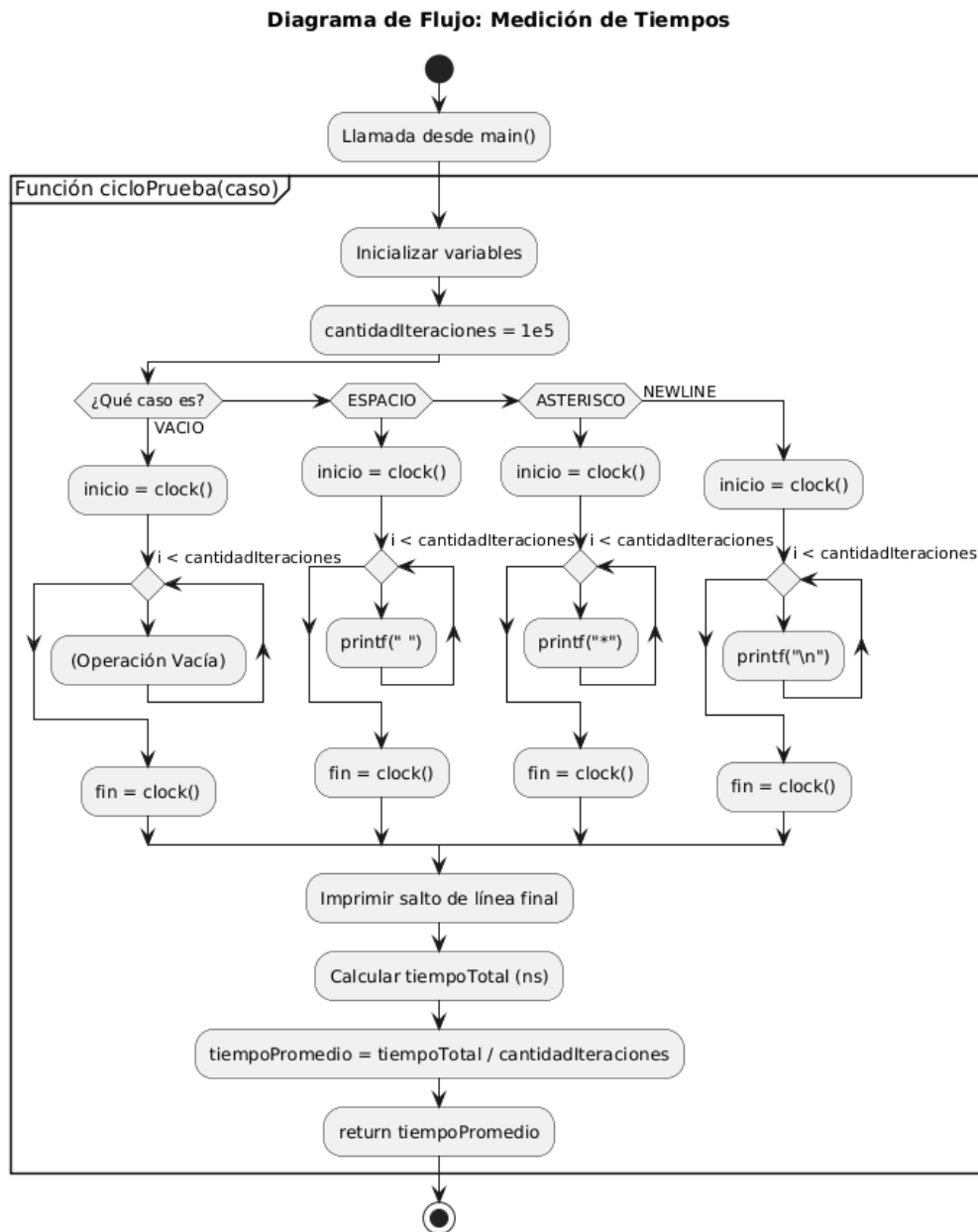


Figura 4: diagrama de flujo (mediciones de tiempo)

1.3. Hito 3

Se utilizó la función `clock()` para medir el tiempo de ejecución, registrando un tiempo inicial antes del ciclo `for` y un tiempo final después de ejecutarlo. Luego se calculó el tiempo total como $((\text{fin} - \text{inicio}) / \text{CLOCKS_PER_SEC})$ y se convirtió a nanosegundos, obteniendo el tiempo promedio dividiendo por (10^8) iteraciones.

caso	[ns]tprom1	[ns]tprom2	[ns]tprom3	[ns]tprom4	[ns]tprom5	[ns]tprom6	[ns]tprom7	[ns]tprom8	[ns]tprom9	[ns]tprom10	promedio	D.Estandar
NADA	2,59	2,43	2,39	1,05	1,09	2,02	2,42	2,31	2,11	2,35	2,076	0,5543204
ESPACIO	21,76	30,04	34,69	32,26	29,63	38,01	31,33	19,66	31,05	20,9	28,933	6,15093917
ASTERISCO	23,54	29,7	33,48	31,76	32,08	37,21	35,75	26,58	33,77	20,58	30,445	5,365197
NEW LINE	1418	2811	2785	2547	2555	2864	2570	1578	2750	1708	2358,6	560,873168

Figura 5: tabla datos (10 ejecuciones)

Para cada caso (VACIO, ESPACIO, ASTERISCO y NEWLINE) se realizaron 10 mediciones. Los resultados muestran promedios de 2,076 ns (VACIO), 28,933 ns (ESPACIO), 30,445 ns (ASTERISCO) y 2358,6 ns (NEWLINE).

El ciclo vacío es el más rápido debido a que no realiza operaciones adicionales. ESPACIO y ASTERISCO presentan tiempos similares a causa del alto costo de printf(). Sin embargo, NEWLINE se presenta un valor mucho mayor ya que imprimir saltos de línea puede provocar vaciado del buffer, aumentando el tiempo y su variabilidad.

Además, se observó que el tiempo total de ejecución depende del computador utilizado, por lo que en algunos equipos (como los de la universidad) fue necesario reducir las iteraciones a (10^5) o (10^4) para evitar tiempos excesivos, aunque la tabla final se obtuvo con (10^8) iteraciones según lo solicitado. Finalmente, se debe incluir el hardware del computador y la versión del compilador utilizado.